

CI/CD Pipelines, Deployment Flow, and Project Environments

Training / Learning Documentation

Prepared by: **SHINY ZERUBBA RAJIVE S**

Date: 12-May-2026

1. Introduction

Modern software development follows a structured process to build, test, deploy, and maintain applications efficiently.

To reduce manual work and improve reliability, organizations use **CI/CD pipelines** along with multiple deployment environments.

These practices help teams:

- Deliver features faster
- Detect bugs early
- Automate deployments
- Maintain application quality
- Reduce production failures

2. What is CI/CD?

CI/CD stands for:

Term	Meaning
CI	Continuous Integration
CD	Continuous Delivery / Continuous Deployment

3. Continuous Integration (CI)

Definition

Continuous Integration is the process where developers frequently merge their code into a shared repository such as:

- [GitHub](#)
- [GitLab](#)
- [Bitbucket](#)

Whenever code is pushed:

- Build process starts automatically
- Automated tests run
- Code quality checks happen
- Errors are identified early

CI Workflow

Developer writes code



Pushes code to Git repository



CI pipeline triggers automatically



Application build starts



Unit tests execute



Code quality/security checks run



Build artifact generated

Benefits of CI

- Early bug detection
- Faster development
- Improved collaboration
- Reduced integration conflicts
- Better code quality

4. Continuous Delivery (CD)

Definition

Continuous Delivery automates the release process after successful CI stages. The application becomes deployment-ready at any time.

Continuous Delivery Flow

Build completed successfully



Application deployed to Test/UAT environment

↓
QA team validates application
↓
Approval provided
↓
Deployment to Production

5. Continuous Deployment

Continuous Deployment is an advanced form of CD where deployment to production happens automatically without manual approval.

Example:

Code Push → Build → Test → Deploy to Production Automatically

Companies like streaming platforms, e-commerce applications, and SaaS products commonly use this approach.

6. What is a CI/CD Pipeline?

A CI/CD pipeline is a sequence of automated stages that move application code from development to production.

Typical CI/CD Pipeline Stages

Code Commit
↓
Build
↓
Unit Testing
↓
Code Analysis
↓
Security Scan
↓
Artifact Creation
↓
Deploy to Dev
↓
Deploy to Test/UAT
↓

Approval



Deploy to Production

7. Tools Used in CI/CD

Purpose	Common Tools
Source Control	Git, GitHub, GitLab
CI/CD Automation	Jenkins, GitHub Actions, Azure DevOps
Containerization	Docker
Orchestration	Kubernetes
Build Tools	Maven, Gradle, npm
Artifact Repository	Nexus, JFrog Artifactory
Monitoring	Prometheus, Grafana

Popular CI/CD platforms include:

- [Jenkins](#)
- [Azure DevOps](#)
- [GitHub Actions](#)
- [GitLab CI/CD](#)

8. How Applications Go Live

The journey from developer laptop to live customer application usually follows this flow:

Step 1: Requirement Gathering

Business team gathers requirements from clients/users.

Example:

- Add payment feature
- Improve dashboard
- Fix bugs

Step 2: Development

Developers write code in local machines.

They work on feature branches:

feature/login-page
feature/payment-api
bugfix/dashboard-error

Step 3: Code Commit

Developers push code to Git repositories.

```
git add .  
git commit -m "Added login feature"  
git push origin feature/login-page
```

Step 4: Pull Request / Code Review

Senior developers review:

- Code quality
- Security
- Performance
- Standards

Only approved code gets merged.

Step 5: CI Pipeline Execution

Once merged:

- Application builds automatically
- Tests execute
- Errors checked
- Deployment package created

Step 6: Deployment to Lower Environments

Application first deploys to:

- Dev Environment
- QA/Test Environment
- UAT Environment

Testing occurs at each stage.

Step 7: Production Deployment

After approvals:

- Application deploys to Production
- Real users access it

This is called **Go Live** or **Release**

9. Different Environments in a Project

Applications are not directly deployed to production.

Different environments are maintained for safe development and testing.

10. Development Environment (DEV)

Purpose

Used by developers for coding and initial testing.

Characteristics

- Frequent code changes
- Less stable
- Debugging enabled
- Used internally only

Activities

- Feature development
- Unit testing
- Initial validation

11. Testing / QA Environment

Purpose

Used by QA testers to validate functionality.

Characteristics

- Stable environment
- Test data used
- Bugs identified here

Activities

- Functional testing
- Integration testing
- Regression testing
- Performance testing

12. UAT Environment (User Acceptance Testing)

Purpose

Business users or clients validate whether the application meets requirements.

Characteristics

- Near-production setup
- Final verification stage

Activities

- Client testing
- Business validation
- Approval before production

13. Staging Environment

Purpose

Acts as a replica of production.

Characteristics

- Same configuration as production
- Final deployment rehearsal

Activities

- Final smoke testing
- Deployment verification
- Production simulation

14. Production Environment (PROD)

Purpose

Live environment used by actual customers.

Characteristics

- Highly secure
- Highly monitored
- Minimal downtime allowed

Activities

- Live user transactions
- Real-time monitoring
- Incident management

15. Environment Flow Diagram

Developer Local Machine



DEV Environment



QA / TEST Environment



UAT Environment



STAGING Environment



PRODUCTION Environment

16. Why Multiple Environments are Important

Benefit	Explanation
Risk Reduction	Prevents faulty code reaching users
Better Testing	Separate validation stages
Stability	Production remains safe
Quality Assurance	Multiple verification checkpoints
Faster Recovery	Easier rollback handling

17. What Happens During Production Deployment

Typical production deployment activities:

- Backup current application
- Stop existing services (if needed)
- Deploy new version
- Run database migrations
- Start services
- Perform smoke testing
- Monitor logs and metrics

18. Rollback Strategy

If deployment fails:

Issue Detected



Rollback Triggered



Previous Stable Version Restored

Rollback minimizes downtime and business impact.

19. Monitoring After Go Live

After deployment, monitoring tools track:

- CPU usage
- Memory usage
- API failures
- Response time
- Error logs
- User activity

Common tools:

- [Grafana](#)
- [Prometheus](#)
- [Datadog](#)

20. Example Real-World Deployment Flow

Developer writes code



Push to GitHub



Jenkins pipeline triggered



Application build created



Automated testing performed



Docker image generated



Deploy to DEV



QA testing completed



UAT approval received



Deploy to Production using Kubernetes



Application goes live